

Working with AI coding tools

AI-generated code stays a proposal until it survives:

AI-generated code stays a proposal until it survives:

- tests

AI-generated code stays a proposal until it survives:

- tests
- review

AI-generated code stays a proposal until it survives:

- tests
- review
- comparison to domain knowledge

AI-generated code stays a proposal until it survives:

- tests
- review
- comparison to domain knowledge
- version-control inspection

AI-generated code stays a proposal until it survives:

- tests
- review
- comparison to domain knowledge
- version-control inspection

AI-generated code stays a proposal until it survives:

- tests
- review
- comparison to domain knowledge
- version-control inspection

Moving faster with a model does not transfer accountability: you still own correctness and what gets merged.

Prompt:

Prompt:

```
generate a python function to compute a multiple linear regression solution  
using linear algebra
```

Prompt:

```
generate a python function to compute a multiple linear regression solution  
using linear algebra
```

I sent this prompt twice to Gemini in separate calls. Both answers looked professional and both needed careful review.

The first response used this core calculation:

The first response used this core calculation:

```
X_T = X.T
```

```
XTX = X_T @ X
```

```
XTX_inv = np.linalg.inv(XTX)
```

```
XTy = X_T @ y
```

```
beta_hat = XTX_inv @ XTy
```

The first response used this core calculation:

```
X_T = X.T  
XTX = X_T @ X  
XTX_inv = np.linalg.inv(XTX)  
XTy = X_T @ y  
beta_hat = XTX_inv @ XTy
```

It also added an intercept by default and returned several extra values (intercept, coefficients, R-squared, full beta vector, predicted values)...

Gemini run 1: normal equation

The first response used this core calculation:

```
X_T = X.T  
XTX = X_T @ X  
XTX_inv = np.linalg.inv(XTX)  
XTy = X_T @ y  
beta_hat = XTX_inv @ XTy
```

It also added an intercept by default and returned several extra values (intercept, coefficients, R-squared, full beta vector, predicted values)...

... more than the narrow wording of the prompt.

The second response avoided explicitly computing the inverse:

The second response avoided explicitly computing the inverse:

```
XTX = X_design.T @ X_design  
XTy = X_design.T @ y  
beta_hat = np.linalg.solve(XTX, XTy)
```

The second response avoided explicitly computing the inverse:

```
XTX = X_design.T @ X_design  
XTy = X_design.T @ y  
beta_hat = np.linalg.solve(XTX, XTy)
```

That is better than `np.linalg.inv`, but it still solves the normal equations, so it can fail when $X.T @ X$ is singular. Could have used more input checking.

Two simplified Gemini-style functions

```
def gemini_regression_inv(X, y, add_intercept=True):
    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float).reshape(-1, 1)
    if add_intercept:
        X = np.hstack((np.ones((X.shape[0], 1)), X))
    beta_hat = np.linalg.inv(X.T @ X) @ X.T @ y
    return beta_hat.flatten()

def gemini_regression_solve(X, y, fit_intercept=True):
    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float)
    if fit_intercept:
        X = np.hstack((np.ones((X.shape[0], 1)), X))
    return np.linalg.solve(X.T @ X, X.T @ y)
```

Four tests

```
class TestGeminiRegression(unittest.TestCase):
    def setUp(self):
        self.x_no_intercept = np.array([[0], [1], [2], [3]], dtype=float)
        self.x_with_intercept = np.array([[1, 0], [1, 1], [1, 2], [1, 3]], dt
        self.y = np.array([1, 3, 5, 7], dtype=float)
    def test_inv_no_intercept(self):
        np.testing.assert_allclose(gemini_regression_inv(
            self.x_no_intercept, self.y), [1, 2])
    def test_inv_already_has_intercept(self):
        np.testing.assert_allclose(gemini_regression_inv(
            self.x_with_intercept, self.y), [1, 2])
    def test_solve_no_intercept(self):
        np.testing.assert_allclose(gemini_regression_solve(
            self.x_no_intercept, self.y), [1, 2])
```

Test output

```
test_inv_already_has_intercept ... ERROR  
test_inv_no_intercept ... ok  
test_solve_already_has_intercept ... ERROR  
test_solve_no_intercept ... ok
```

```
numpy.linalg.LinAlgError: Singular matrix
```

```
Ran 4 tests in 0.017s
```

```
FAILED (errors=2)
```

Test output

```
test_inv_already_has_intercept ... ERROR  
test_inv_no_intercept ... ok  
test_solve_already_has_intercept ... ERROR  
test_solve_no_intercept ... ok
```

```
numpy.linalg.LinAlgError: Singular matrix
```

```
Ran 4 tests in 0.017s
```

```
FAILED (errors=2)
```

Gemini can write code just fine, but plausible code still carries assumptions that only review and tests can surface.

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

Its generated suite did some useful things:

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

Its generated suite did some useful things:

- compared against `scikit-learn`

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

Its generated suite did some useful things:

- compared against `scikit-learn`
- used `np.allclose`

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

Its generated suite did some useful things:

- compared against `scikit-learn`
- used `np.allclose`
- checked singular-matrix behavior

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

Its generated suite did some useful things:

- compared against `scikit-learn`
- used `np.allclose`
- checked singular-matrix behavior

The happy-path testing failure

I also asked Gemini to write tests for the regression functions.

Its generated suite did some useful things:

- compared against `scikit-learn`
- used `np.allclose`
- checked singular-matrix behavior

But it also made the problem easier for itself: the suite tested the functions on the model's own assumptions instead of the behavior we actually wanted.

What happy-path tests look like

Gemini wrote tests like this:

What happy-path tests look like

Gemini wrote tests like this:

```
beta_a = linear_regression_A(  
    self.X_no_intercept,  
    self.y_no_intercept,  
    fit_intercept=False,  
)  
self.assertTrue(np.allclose(beta_a, expected_beta_sk_no_intercept))
```

What happy-path tests look like

Gemini wrote tests like this:

```
beta_a = linear_regression_A(  
    self.X_no_intercept,  
    self.y_no_intercept,  
    fit_intercept=False,  
)  
self.assertTrue(np.allclose(beta_a, expected_beta_sk_no_intercept))
```

This is not useless, but it sidesteps the question that exposed the problem: what if the caller gives a design matrix that already has an intercept column?

Rule: write the test first

- If the model writes both the code and the test suite, it will generally make itself look good

Rule: write the test first

- If the model writes both the code and the test suite, it will generally make itself look good
- You can keep control by writing tests first and forcing the model to satisfy your contract

Rule: write the test first

- If the model writes both the code and the test suite, it will generally make itself look good
- You can keep control by writing tests first and forcing the model to satisfy your contract
- Test-driven development becomes more important with AI coding tools

For an LLM, **context** is everything currently available to the model:

For an LLM, **context** is everything currently available to the model:

- your prompt

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages
- attached files

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages
- attached files
- tool outputs

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages
- attached files
- tool outputs
- error messages

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages
- attached files
- tool outputs
- error messages
- system and project instructions

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages
- attached files
- tool outputs
- error messages
- system and project instructions

For an LLM, **context** is everything currently available to the model:

- your prompt
- earlier messages
- attached files
- tool outputs
- error messages
- system and project instructions

The model's next output is conditioned on this context, and when that context is wrong, stale, or cluttered, the output suffers.

Context engineering is often more useful than hunting for magic prompt phrases.

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files
- strip irrelevant or wrong material

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files
- strip irrelevant or wrong material
- state constraints explicitly

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files
- strip irrelevant or wrong material
- state constraints explicitly
- keep durable project knowledge outside the chat

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files
- strip irrelevant or wrong material
- state constraints explicitly
- keep durable project knowledge outside the chat
- restart when the conversation becomes polluted

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files
- strip irrelevant or wrong material
- state constraints explicitly
- keep durable project knowledge outside the chat
- restart when the conversation becomes polluted

Context engineering is often more useful than hunting for magic prompt phrases.

Useful habits:

- give the model the right files
- strip irrelevant or wrong material
- state constraints explicitly
- keep durable project knowledge outside the chat
- restart when the conversation becomes polluted

The best prompt cannot compensate for bad context.

Large context windows are useful, but they are not an excuse to skip pruning.

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements
- over-attend to recent failures

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements
- over-attend to recent failures
- blend incompatible instructions

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements
- over-attend to recent failures
- blend incompatible instructions
- imitate obsolete code from earlier attempts

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements
- over-attend to recent failures
- blend incompatible instructions
- imitate obsolete code from earlier attempts
- forget project-level constraints

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements
- over-attend to recent failures
- blend incompatible instructions
- imitate obsolete code from earlier attempts
- forget project-level constraints

Large context windows are useful, but they are not an excuse to skip pruning.

As context grows, models can:

- miss earlier requirements
- over-attend to recent failures
- blend incompatible instructions
- imitate obsolete code from earlier attempts
- forget project-level constraints

Keep the active context as small as the task allows.

Worked example: in-context learning

Prompt A:

Worked example: in-context learning

Prompt A:

```
generate a python function to compute a multiple linear regression solution  
using linear algebra
```


Worked example: in-context learning

Prompt A:

`generate a python function to compute a multiple linear regression solution
using linear algebra`

Prompt B:

Worked example: in-context learning

Prompt A:

`generate a python function to compute a multiple linear regression solution using linear algebra`

Prompt B:

`why are type hints important when creating a python function?`

`generate a python function to compute a multiple linear regression solution using linear algebra`

Worked example: in-context learning

Prompt A:

`generate a python function to compute a multiple linear regression solution using linear algebra`

Prompt B:

`why are type hints important when creating a python function?`

`generate a python function to compute a multiple linear regression solution using linear algebra`

Adding context, even obliquely, can steer the model. Prompt B will usually produce type hints even when the regression prompt alone does not mention them.

Context steers the model, good or bad!

In-context learning cuts both ways

Context steers the model, good or bad!

If the model spent many turns on a bad strategy, it may keep imitating that strategy.

In-context learning cuts both ways

Context steers the model, good or bad!

If the model spent many turns on a bad strategy, it may keep imitating that strategy.

Usually the right strategy is:

Context steers the model, good or bad!

If the model spent many turns on a bad strategy, it may keep imitating that strategy.

Usually the right strategy is:

1. stop

Context steers the model, good or bad!

If the model spent many turns on a bad strategy, it may keep imitating that strategy.

Usually the right strategy is:

1. stop
2. summarize what was learned and what was good

Context steers the model, good or bad!

If the model spent many turns on a bad strategy, it may keep imitating that strategy.

Usually the right strategy is:

1. stop
2. summarize what was learned and what was good
3. clear the context

In-context learning cuts both ways

Context steers the model, good or bad!

If the model spent many turns on a bad strategy, it may keep imitating that strategy.

Usually the right strategy is:

1. stop
2. summarize what was learned and what was good
3. clear the context
4. restart with a cleaner prompt that includes the curated summary

Examples can be more effective than abstract instructions: they define the style and shape of the answer you want.

Examples can be more effective than abstract instructions: they define the style and shape of the answer you want.

Java: `public List<String> filterUserNames(List<User> users, int minAge, boolean activeOnly)`

C++: `std::vector<std::string> filter_user_names(const std::vector<User>& users, int min_age, bool active_only)`

Rust: `fn filter_user_names(users: &[User], min_age: u32, active_only: bool) -> Vec<String>`

Haskell: `filterUserNames :: [User] -> Int -> Bool -> [String]`

Python: ?

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

Trade-offs:

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

Trade-offs:

- slower responses

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

Trade-offs:

- slower responses
- more tokens

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

Trade-offs:

- slower responses
- more tokens
- sometimes over-complicated answers

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

Trade-offs:

- slower responses
- more tokens
- sometimes over-complicated answers

For hard debugging or design tasks, asking the model to reason more carefully can improve performance on some problems.

Trade-offs:

- slower responses
- more tokens
- sometimes over-complicated answers

Reserve this for genuinely hard problems, not every autocomplete.

Tasks that are **too large** tend to lose details, produce inconsistent code, and bury reviewers.

Tasks that are **too large** tend to lose details, produce inconsistent code, and bury reviewers.

Tasks that are **too small** burn time on prompting and may never fit together.

Tasks that are **too large** tend to lose details, produce inconsistent code, and bury reviewers.

Tasks that are **too small** burn time on prompting and may never fit together.

Aim for work units that are still meaningful but small enough to test and review cleanly.

Persistent context has different files for different jobs

There are three different things people often mix together:

Persistent context has different files for different jobs

There are three different things people often mix together:

File type	Job
Agent/project instructions	Tell the agent how to behave in this project
Memory/planning files	Tell the agent what the project is doing now
Skills	Teach the agent a reusable workflow

Persistent context has different files for different jobs

There are three different things people often mix together:

File type	Job
Agent/project instructions	Tell the agent how to behave in this project
Memory/planning files	Tell the agent what the project is doing now
Skills	Teach the agent a reusable workflow

Keeping these separate makes instructions easier to maintain.

Role: define how the agent must behave in this codebase—style rules, forbidden actions, required commands, and what “done” means.

Role: define how the agent must behave in this codebase—style rules, forbidden actions, required commands, and what “done” means.

Where they live:

Role: define how the agent must behave in this codebase—style rules, forbidden actions, required commands, and what “done” means.

Where they live:

Tool	Conventional path
OpenAI Codex / ChatGPT	<code>AGENTS.md</code> at repo root
Cursor	<code>.cursor/rules/*.mdc</code>
Claude Code	<code>CLAUDE.md</code> at repo root (or any ancestor directory)
Generic	<code>AGENTS.md</code> or <code>.ai/instructions.md</code>

Best practices:

Best practices:

- **Obtain:** let the agent draft a first version by describing your project; review and tighten it.

Best practices:

- **Obtain:** let the agent draft a first version by describing your project; review and tighten it.
- **Create:** keep it short—under 200 lines; use headers and bullet lists so the model can scan fast.

Best practices:

- **Obtain:** let the agent draft a first version by describing your project; review and tighten it.
- **Create:** keep it short—under 200 lines; use headers and bullet lists so the model can scan fast.
- **Maintain:** update after every significant architectural decision; version-control it like code; note non-negotiables explicitly.

Role: carry the project's current state across sessions—what was decided, what tasks remain, what has already failed.

Role: carry the project's current state across sessions—what was decided, what tasks remain, what has already failed.

Where they live: repo root or a **docs/** subdirectory; names are conventional, not enforced.

Role: carry the project's current state across sessions—what was decided, what tasks remain, what has already failed.

Where they live: repo root or a **docs/** subdirectory; names are conventional, not enforced.

File	Typical content
PRD.md / PLANNING.md	Goals, architecture, constraints
TASKS.md	Current work plan, status markers
SCRATCHPAD.md	Ephemeral notes, dead ends, scratch thoughts

Best practices:

Best practices:

- **Obtain:** ask the agent to generate a `TASKS.md` from your planning notes at the start of a session.

Best practices:

- **Obtain:** ask the agent to generate a `TASKS.md` from your planning notes at the start of a session.
- **Create:** separate long-lived decisions (PRD) from volatile state (tasks, scratchpad).

Best practices:

- **Obtain:** ask the agent to generate a `TASKS.md` from your planning notes at the start of a session.
- **Create:** separate long-lived decisions (PRD) from volatile state (tasks, scratchpad).
- **Maintain:** update task status before clearing context; archive completed items rather than deleting, so the record of decisions survives.

Role: package a reusable, step-by-step procedure so the agent performs a class of task the same way every time, across projects.

Role: package a reusable, step-by-step procedure so the agent performs a class of task the same way every time, across projects.

Where they live:

Role: package a reusable, step-by-step procedure so the agent performs a class of task the same way every time, across projects.

Where they live:

Scope	Path
Per-project	<code>.cursor/skills/<name>/SKILL.md</code>
User-wide	<code>~/.cursor/skills/<name>/SKILL.md</code>
Generic	<code>.agents/skills/<name>/SKILL.md</code> or <code>~/.agents/skills/<name>/SKILL.md</code>

Role: package a reusable, step-by-step procedure so the agent performs a class of task the same way every time, across projects.

Where they live:

Scope	Path
Per-project	<code>.cursor/skills/<name>/SKILL.md</code>
User-wide	<code>~/.cursor/skills/<name>/SKILL.md</code>
Generic	<code>.agents/skills/<name>/SKILL.md</code> or <code>~/.agents/skills/<name>/SKILL.md</code>

Skills can ship with `scripts/`, `references/`, and `assets/` sub-folders alongside the `.md`.

Best practices:

Best practices:

- **Obtain:** extract repeated ad-hoc prompts into a skill file; community skill libraries exist for common workflows.

Best practices:

- **Obtain:** extract repeated ad-hoc prompts into a skill file; community skill libraries exist for common workflows.
- **Create:** one skill per task type; use a **paths:** glob to scope it so only relevant files trigger it.

Best practices:

- **Obtain:** extract repeated ad-hoc prompts into a skill file; community skill libraries exist for common workflows.
- **Create:** one skill per task type; use a **paths:** glob to scope it so only relevant files trigger it.
- **Maintain:** treat as code—version-control, test after tool updates, retire stale skills rather than leaving them to pollute context.

Agent instructions

Code style (NON-NEGOTIABLE)

- Prefer small functions with clear names.
- Preserve existing public APIs unless asked to change them.

Testing (NON-NEGOTIABLE)

- Use ``unittest``, not ``pytest``, in this project.
- Write or update tests before implementation.
- Never relax a test just to make generated code pass.

Workflow

- Run ``python -m unittest discover -s tests`` before reporting success.
- Use small commits at working checkpoints.

name: *python-formatting*

description: *Format new or edited Python like the course examples (PEP 8, typ*

paths: *"**/*.py"*

Python formatting

Use ``from __future__ import annotations`` in new modules. Type-hint public APIs; prefer ``pathlib.Path`` over ``os.path``; use f-strings.

Follow PEP 8 (four-space indent, sensible wraps). No star imports; avoid mutable default arguments; keep functions short with descriptive names.

File	Use it for	Lifespan
AGENTS.md	Project constitution and constraints	Project-wide
TASKS.md	Current work plan and status	Changes often
SCRATCHPAD.md	Temporary notes and failed attempts	Clean regularly
SKILL.md	Reusable task procedure	Across projects

File	Use it for	Lifespan
AGENTS.md	Project constitution and constraints	Project-wide
TASKS.md	Current work plan and status	Changes often
SCRATCHPAD.md	Temporary notes and failed attempts	Clean regularly
SKILL.md	Reusable task procedure	Across projects

Do not make every instruction global: put each instruction where it belongs.

During agentic coding, manage context deliberately: inspect what the agent has loaded, compact when you are mid-task but context is bloated, clear when you are at a breakpoint, reload durable files after clearing, and commit working states before risky changes. The model has no separate sense of what matters; only the context you give it carries that signal.

During agentic coding, manage context deliberately: inspect what the agent has loaded, compact when you are mid-task but context is bloated, clear when you are at a breakpoint, reload durable files after clearing, and commit working states before risky changes. The model has no separate sense of what matters; only the context you give it carries that signal.

Commit-clear-reload

During agentic coding, manage context deliberately: inspect what the agent has loaded, compact when you are mid-task but context is bloated, clear when you are at a breakpoint, reload durable files after clearing, and commit working states before risky changes. The model has no separate sense of what matters; only the context you give it carries that signal.

Commit-clear-reload

A useful rhythm: review the diff, update task notes or scratchpad, commit the working state, clear or compact the context, then reload instruction and memory files. That sequence gives both you and the model a clean checkpoint.